# *MedStorePro – Project Proposal Document (Medical Store Management System)*

***Areeba Mahnoor  BSSE23033***

***Zainab Faisal  BSSE23045***

## 1. Executive Summary

MedStorePro is a cloud-ready, full-stack medical store/pharmacy management platform designed to digitize and automate core retail-pharmacy operations such as inventory control, sales/order processing, supplier handling, and administrative user management. The platform provides a modern dashboard with analytics and real-time operational views to help staff reduce stock-outs, prevent expiry losses, and improve order handling efficiency. Technology Stack

- **Frontend:** React 19 + Material UI (MUI), React Router, Axios
- **Backend:** Node.js + Express REST API
- **Database:** MongoDB (Mongoose ODM)
- **Security:** JWT authentication, role-based access control, Helmet, CORS, rate limiting

**Intelligent / Automation Features**

- Low-stock and expiry monitoring (threshold-based inventory alerts, expiring inventory views)
- Automated stock deduction/restoration on order status changes (e.g., confirm/cancel)
- Scheduled data export to S3 using AWS Lambda (e.g., daily export of orders/products/users)

**Deployment Approach (AWS Free Tier friendly)**

- **Frontend hosting:** AWS Amplify (static React build)
- **API hosting:** AWS Lambda + API Gateway (serverless REST API) or EC2
- **Data exports:** AWS Lambda → S3
- **Handle incoming traffic:** Application Load Balancer (ALB) and attach it to an Auto Scaling Group (ASG)
- **Database:** MongoDB (local/self-hosted or Atlas based on constraints)

---

## 2. Introduction

Medical stores and pharmacies manage time-sensitive, regulated inventory where stock availability, expiry tracking, and accurate order processing directly impact patient outcomes and

business performance. Many small-to-mid setups still rely on manual stock tracking and fragmented workflows, leading to frequent issues such as overselling, expired product losses, inconsistent supplier records, and limited reporting. MedStorePro addresses these limitations with a unified digital platform. It provides a secure login system, role-based permissions (admin/manager/user), and a dashboard that consolidates inventory and order insights in a single interface. The solution is designed to be deployable within an AWS Free Tier–focused setup while still following cloud-native principles.

## 3. Problem Statement

Current pharmacy/medical store operations commonly face:

- Manual inventory tracking, causing incorrect quantities and frequent stock-outs.
- Expiry risk, where products expire unnoticed due to lack of automated monitoring and alerts.
- Inefficient order workflows, including inconsistent order status handling and missing stock synchronization.
- Limited reporting and analytics, making it difficult to evaluate revenue trends, sales performance, and inventory valuation.
- Data backup/export challenges, where businesses lack reliable, automated export pipelines for reporting and audit needs.

These challenges create operational delays, financial loss, and unreliable decision-making. MedStorePro proposes to solve these issues using secure, scalable full-stack architecture with optional cloud automation for backups and exports.

## 4. Aim & Objectives

### *Aim:*

To design, implement, and deploy a secure, scalable medical store management system that supports inventory accuracy, efficient order processing, analytics, and automated exports using AWS services.

### *Objectives:*

- Develop a full-stack application using React, Node.js/Express, and MongoDB.
- Implement secure authentication and authorization using JWT and role-based access control (Admin/Manager/User).
- Build inventory management features including product CRUD, stock adjustments (restock), low-stock detection, and expiry tracking.
- Develop order management workflows with status transitions and stock synchronization.

- Provide analytics and reporting for dashboard KPIs (orders, products, suppliers, revenue trends).
- Enable automated export pipelines (MongoDB → AWS Lambda → S3) for backups/reporting using AWS Free Tier–friendly services and patterns.

## 5. System Architecture Overview

This architecture shows a pharmacy management system with **two main flows**:

- **Operational flow:** Users access the React frontend hosted on **AWS Amplify** (deployed from GitHub). The frontend sends requests to the backend API (shown via **API Gateway**) which is handled by the **Node/Express backend on EC2**, storing and retrieving data from **MongoDB (Atlas)**.

- **Analytics flow:** A **Lambda export function** running inside a **VPC private subnet** connects to MongoDB using outbound internet via **NAT Gateway** + **Internet Gateway**, exports collections to **Amazon S3**, and then **AWS Glue Crawler/Data Catalog** prepares tables so **Amazon Athena** can run SQL queries on the exported S3 data (with results stored back in S3).